Министерство науки и высшего образования РФ

Федеральное государственное автономное образовательное учреждение высшего образования «Национальный исследовательский технологический университет «МИСиС»

Институт ИТКН

Кафедра АСУ

Курс «Основы теории информации»

Практическая Работа №1

Выполнил: студента группы БИВТ-24-СП-4 (ОТИ-11)

Бахров Антон Иванович

Проверил: Александр Кривяков

г. Москва

2026 год

Ссылка на репозиторий в GitHub:

https://github.com/BahrovAnton249/VK_prakticheskie

1)Вариант и масштаб:

Поле	Значение
ФИО (как в VK Education	Бахров Антон Иванович
Вариант (запрос)	Q18 (3-й вариант)
Масштаб	sf0.1
Почему выбрал такой масштаб	Я выбрал sf0.1, потому что он достаточен для демонстрации медальона и не перегружает ноутбук. Масштаб sf1 (6 млн) приводил к снижению отзывчивости системы и последующему падению Trino с последующей ошибкой в DBeaver. Поэтому я предпочел масштаб sf0.1 масштабу sf1 или выше.

2)Поднятые сервисы — и почему именно они

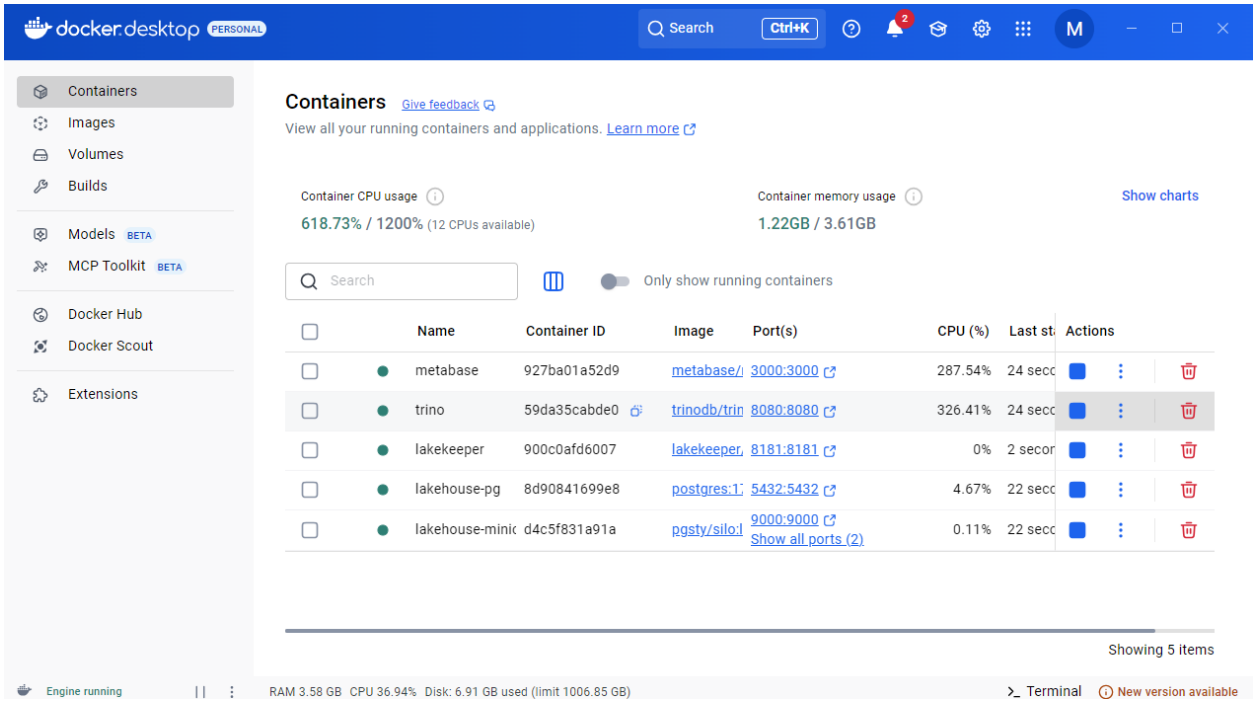


Рисунок 1 Поднятые сервисы



Рисунок 2 Поднятые сервисы через команду docker ps

Сервис	**Роль**	**Почему именно он**
MinIO (Silo)	S3-совместимое объектное хранилище (сюда физически падают parquet-файлы)	Открытый, S3-совместимый, бесплатный, работает локально в Docker.
PostgreSQL	БД, где Lakekeeper хранит метаданные каталога. Метаданные Lakekeeper.	Надёжная ACID-СУБД, стандарт для хранения каталога
Lakekeeper	Iceberg REST Catalog — каталог метаданных таблиц.	Open-source, REST API, поддерживает multi-warehouse
Trino	SQL-движок. Ходит в Lakekeeper за метаданными и читает/пишет данные в MinIO.	Быстрый, коннектор Iceberg, федеративные запросы
BI Superset: Metabase	Metabase — это BI-инструмент, который визуализирует данные lakehouse. Он читает данные через Trino и строит дашборды, графики, отчёты для бизнес-пользователей.	Он был выбран мной из-за простоты в использовании, низких потребностей и скорости работы

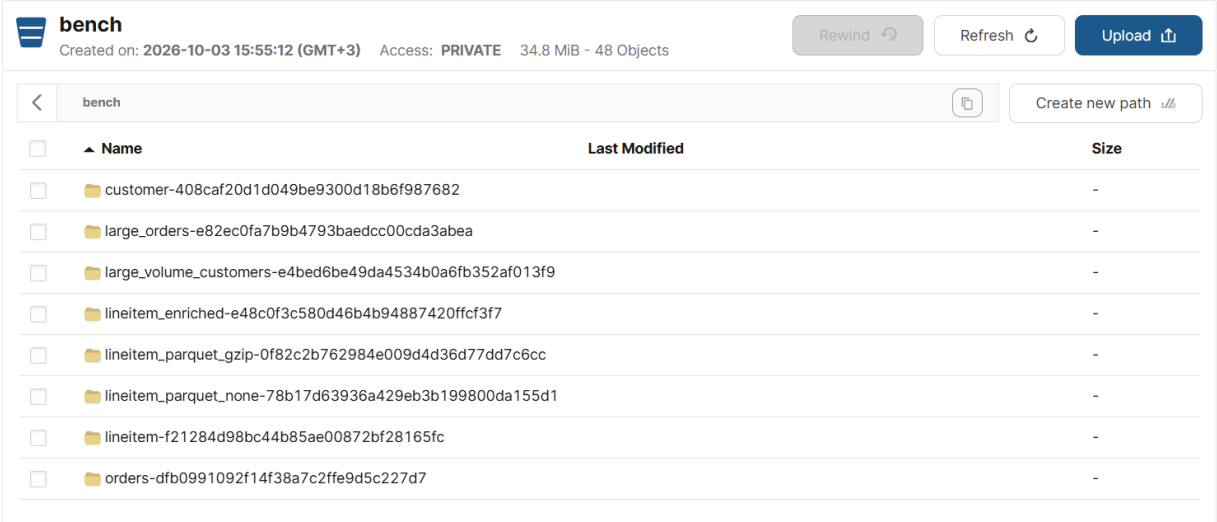


Рисунок 3 MinIO (Silo)

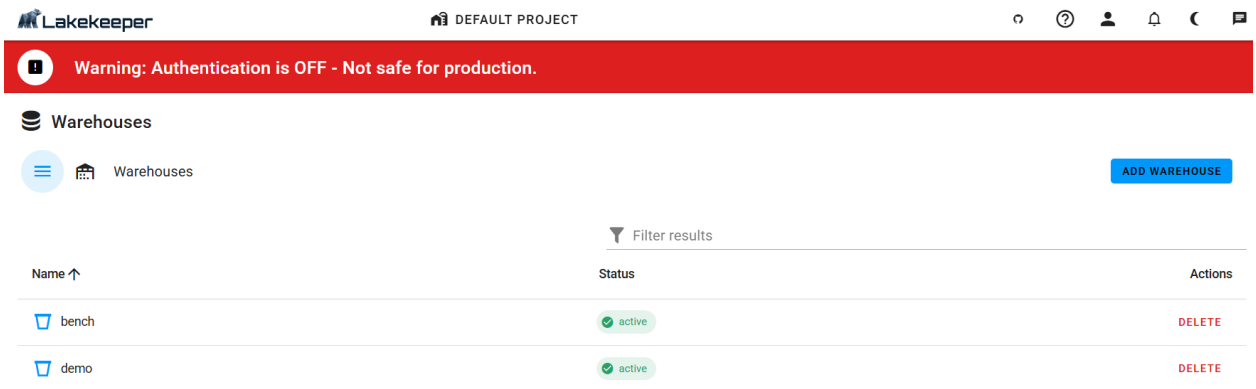


Рисунок 4 Warehouse

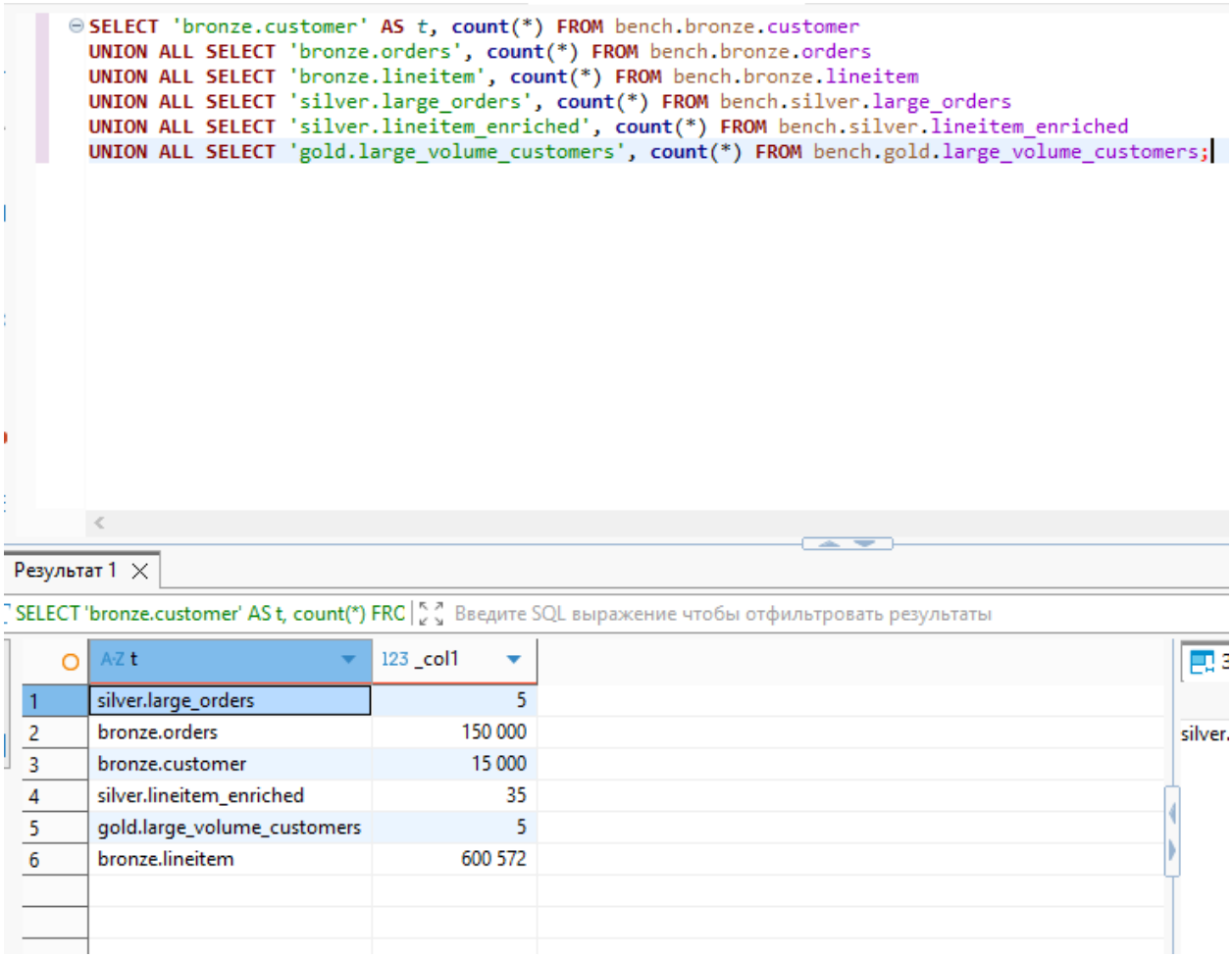


Рисунок 5 Счётчики медальона

Скрипты, которыми поднимал стек:

Использовал docker start, потому что docker run поднимает стек создавая контейнер, а docker start возобновляет уже созданный ранее.

```bash
docker start lakehouse-minio...lakehouse-minio # MinIO — объектное хранилище
docker start lakehouse-pg ...lakehouse-pg # PostgreSQL — метаданные Lakekeeper
docker start lakekeeper...lakekeeper # Lakekeeper — Iceberg REST Catalog
docker start trino...trino # Trino — SQL-движок
```

Но вот скрипты которые я использовал по ходу выполнения лабораторных работ:

Создание сети

docker network create lakehouse-net

MinIO — S3-хранилище

docker run -d --name lakehouse-minio -p 9000:9000 -p 9001:9001 -e "MINIO_ROOT_USER=minioadmin" -e "MINIO_ROOT_PASSWORD=minioadmin" -v minio-data:/data pgsty/silo:latest server /data --console-address ":9001"

PostgreSQL — метаданные

docker run -d --name lakehouse-pg -e "POSTGRES_PASSWORD=postgres" -p 5432:5432 -v pg-data:/var/lib/postgresql/data postgres:17

Lakekeeper

docker run --rm -e "LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!" -e "LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@lakehouse-pg:5432/postgres" -e "LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@lakehouse-pg:5432/postgres" quay.io/lakekeeper/catalog:latest-main migrate

Lakekeeper — REST-каталог

docker run -d --name lakekeeper --network lakehouse-net -e "LAKEKEEPER__PG_ENCRYPTION_KEY=This-is-NOT-Secure!" -e "LAKEKEEPER__PG_DATABASE_URL_READ=postgresql://postgres:postgres@lakehouse-pg:5432/postgres" -e "LAKEKEEPER__PG_DATABASE_URL_WRITE=postgresql://postgres:postgres@lakehouse-pg:5432/postgres" -p 8181:8181 quay.io/lakekeeper/catalog:latest-main serve

Trino — SQL-движок

docker run -d --name trino --network lakehouse-net -p 8080:8080 -v "$HOME\lakehouse\catalog:/etc/trino/catalog" trinodb/trino:476

3) Разбор запроса — какие данные нужны.

Таблица | Текст

	A-Z name	123 custkey	123 orderkey	orderdate	123 totalprice	123 tota
1	Customer#000001639	1 639	502 886	1994-04-12	456 423,88	
2	Customer#000006655	6 655	29 158	1995-10-21	452 805,02	
3	Customer#000014110	14 110	565 574	1995-09-24	425 099,85	
4	Customer#000001775	1 775	6 882	1997-04-09	408 368,1	
5	Customer#000011459	11 459	551 136	1993-05-19	386 812,74	

Таблица TPC-H	Нужные колонки	Что запрос с ними делает
customer	custkey, name	Join с orders по custkey; вывод name

Таблица TPC-H	Нужные колонки	Что запрос с ними делает
orders	orderkey, custkey, orderdate, total price	Join с customer и lineitem; фильтр orderkey IN (подзапрос); вывод orderdate, totalprice; сортировка по Total price DESC
lineitem	orderkey, quantity	Join с orders; подзапрос GROUP BY orderkey HAVING sum(quantity) > 300; агрегат SUM(quantity)

4) Медальон — решения по каждому слою.

Bronze (сырой):

Таблица	Колонки	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
customer	все	15 000	0.8	PARQUET	GZIP	нет	Сырые данные из TPC-H без изменений
orders	все	150 000	3.4	PARQUET	GZIP	нет	Сырые данные из TPC-H без изменений
lineitem	все	600 572	14.1	PARQUET	GZIP	нет	Сырые данные из TPC-H без изменений

Сырые данные из TPC-H без изменений — минимальная обработка при загрузке. Формат Parquet — колоночный, эффективен для аналитики, стандарт де-факто. Сжатие GZIP — сильное сжатие, экономит место в MinIO. Партиционирование не использовал — на масштабе sf0.1 таблицы маленькие (< 20 МБ), партиции не дают выигрыша.

Почему Parquet?

Я выбрал Parquet потому что это колоночный формат хранения.

В аналитических нагрузках запросы читают несколько колонок из многих — например, Q18 использует orderkey, quantity, custkey, name, orderdate, totalprice. В колоночном формате нужные колонки читаются отдельно от остальных — это в разы быстрее, чем в строчном формате (CSV, Avro). Кроме

того, Parquet является стандартным выбором в data-инжиниринге: с ним работают Trino, Spark, Hive, Metabase — все инструменты моего стека.
Он самодокументируемый — хранит схему, типы колонок, статистику по блокам.

Почему GZIP?

GZIP — сильный кодек сжатия. На данных TPC-H он даёт сжатие 3–5× относительно NONE. Я выбрал его из-за его эконмоности памяти, скорости сжатия и распаковки, которая больше чем, например, у ZSTD на малых блоках. Также он был выбран мной из-за широкой совместимости. Его поддерживает как и Trino, так и Spark.
Альтернативы: SNAPPY — быстрее, но сжимает слабее; ZSTD — сильнее, но не всегда быстрее GZIP; NONE — быстро, но 3–5× больше места. Для учебной задачи GZIP — оптимальный компромисс.

Почему без партиционирования?

Партиционирование разбивает таблицу на подпапки по значению колонки — например, year=1994/, year=1995/. Это ускоряет фильтры по партиционной колонке (Trino читает только нужные папки).
Я его не использовал ,потому что на масштабе sf0.1 таблицы маленькие, а партиции дают микроскопический выигрыш, что усложняют структуру. Также если партиционировать 600 тыс. строк по 7 годам, получится ~85 тыс. строк на партицию — это маленькие файлы, которые тормозят планирование, что стало еще одним признаком того, что лучше использовать партиционирование.

Почему получились такие значения объема?

Bronze содержит сырые данные TPC-H без изменений — поэтому объём пропорционален масштабу.Соотношение размеров соответствует TPC-H: lineitem в 4–5 раз больше orders, а orders в 4–5 раз больше customer — это структура бенчмарка. lineitem содержит позиции заказов (в среднем ~4 позиции на заказ), поэтому строк больше всего.
Объём после сжатия GZIP — в 3–5 раз меньше сырого. Без сжатия lineitem был бы ~50–70 МБ.

Silver (очищенный):

Таблица	Колонки	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
large_orders	orderkey	5	0.000195	PARQUET	GZIP	нет	Только крупные заказы (quantity > 300)

lineitem_enriched	orderkey, quantity, extendedprice,discount, revenue	35	0.00156	PARQUET	GZIP	нет	Обогащённые строки с revenue

Отфильтрованные крупные заказы (сумма quantity > 300) и обогащённые строки с вычисленной revenue. Формат Parquet, сжатие GZIP — те же соображения.

Партиционирование не использовал — итоговые таблицы крошечные (5 и 35 строк).

Почему Parquet?

Те же причины, что в Bronze: Parquet — колоночный формат, GZIP — сильное сжатие. Консистентность медальона — все слои в одном формате, значит, нет лишних конвертаций при чтении Trino.

Почему GZIP?

Те же причины, что в Bronze: Parquet — колоночный формат, GZIP — сильное сжатие. Консистентность медальона — все слои в одном формате, значит, нет лишних конвертаций при чтении Trino.

Почему без партиционирования?

Silver large_orders содержит 5 строк, lineitem_enriched — 35 строк. Партиционирование бессмысленно т.к. вся таблица влезает в одну партицию.

Почему получились такие значения объема?

«Silver содержит не все данные, а только отфильтрованные. large_orders — только orderkey, где сумма quantity > 300. Из 600 тыс. строк lineitem таких заказов оказалось 5 — это редкое событие, поэтому таблица крошечная (0.2 КБ). lineitem_enriched — все строки этих крупных заказов (все позиции). Их 35 — потому что 5 заказов × ~7 позиций = 35. Плюс добавлена обогащённая колонка revenue = extendedprice × (1 − discount).

Поэтому не удивительно, что вес в МБ получаются таким незначительным.

Gold (витрина):

Таблица	Колонки	Строк	Вес, МБ	Формат	Сжатие	Партиционирование	Обоснование
large_volume_customers	name, custkey,orderkey orderdate, totalprice,	5	0.00107	PARQUET	GZIP	нет	Топ 100 клиентов по сумме заказа

	total_quantity						

Готовый ответ на Q18 — витрина для бизнес-пользователя. Формат Parquet, сжатие GZIP. Партиционирование не использовал — 5 строк, одна партиция. Для реальной аналитики на больших объёмах можно добавить партиционирование по ship_year или returnflag.

Почему Parquet?

Gold — витрина для бизнеса. Использую формат Parquet, потому что Metabase и Trino читают Parquet нативно, то есть напрямую, используя встроенные, «родные» для платформы механизмы.

Почему GZIP?

Gold — витрина для бизнеса. Использую формат Parquet, потому что Metabase и Trino читают Parquet нативно, то есть напрямую, используя встроенные, «родные» для платформы механизмы.

Почему без партиционирования?

Gold large_volume_customers — 5 строк. Партиционировать 5 строк — нет смысла. В реальной аналитике на миллионах строк партиционируют по ship_year или region, но здесь таблица тривиально мала.

Почему получились такие значения объема?

Gold содержит ответ на Q18 — топ клиентов по сумме заказа с общим количеством товара > 300. Результат — 5 строк (по числу крупных заказов). Каждая строка — 6 полей (name, custkey, orderkey, orderdate, totalprice, total_quantity). Это финальный агрегат — от 600 тыс. строк сырых данных до 5 строк бизнес-ответа. Уменьшение в 120 000 раз — типичный эффект медальона, вполне ожидаемо что веса в МБ такие маленькие.

5) Тесты производительности.

Сжатие на Bronze	**Время запроса**	**Размер Bronze, МБ**	**Твой вывод (что лучше и почему)**
GZIP	17.3569720	14.1	GZIP быстрее на этом объёме
ZSTD	25.5184074	14.1	STD медленнее, но размер тот же

У GZIP и ZSTD получилось одинаковое значение — 14.1 МБ.
Это ожидаемо потому что оба кодека относятся к классу сильных кодеков общего назначения, и на текстово-числовых данных TPC-H они дают близкий коэффициент сжатия.
На масштабе sf0.1 (14 МБ) разница в 1–5 % теряется в округлении до десятых долей МБ.
На больших объёмах (sf1, sf10) ZSTD обычно выигрывает у GZIP на 5–15 % по размеру —
но на sf0.1 этот эффект не проявляется.
Время запроса Q18:
GZIP — 17.4 с, ZSTD — 25.5 с. GZIP быстрее примерно в 1.5 раза.
Это не совсем стандартный результат: теоретически ZSTD должен быть быстрее GZIP,
потому что использует более современный алгоритм (окно сжатия больше, энтропийное
кодирование эффективнее). На практике разница может быть объяснена тремя факторами: Малый объём данных, условия замера и реализация ZSTD в Trino 476.
Можно подитожить, что на масштабе sf0.1 разница между GZIP и ZSTD по размеру незаметна — оба дают около 14 МБ.
По скорости GZIP оказался быстрее (17.4 с vs 25.5 с), но этот результат нельзя считать окончательным — он получен в разных условиях и на малом объёме.
Для корректного сравнения желательно прогнать замеры несколько раз в одних условиях и с большим масштабом, например sf1 или выше, но по ранее описанной мною причине, выбрать масштаб выше не получится.
Для малых и средних таблиц (мб) — разница между GZIP и ZSTD несущественна, можно использовать любой, а для больших таблиц (гигабайты и выше) — предпочтительнее ZSTD, т.к. он дает тот же размер при меньших затратах CPU на распаковку, что ускоряет аналитические запросы. GZIP оправдан для архивных данных, которые читаются редко — там его максимальное сжатие важнее скорости.

6. Часть 2 — осмысление и работа с инструментами.

6.1. Понять и объяснить:

Вопрос	Твой ответ
Зачем нужны большие данные и lakehouse (а не обычная БД/хранилище)?	Обычная реляционная БД не масштабируется на петабайты: вертикальное масштабирование упирается в железо, горизонтальное — дорого и сложно (шардирование, репликация, транзакции между узлами). Data lake дёшев и безгранично масштабируется, но не даёт ACID, схемы и SQL — данные превращаются в «болото». Lakehouse объединяет оба мира: хранение в объектном хранилище, табличный формат с ACID (Iceberg), SQL-доступ (Trino). Итог: дёшево как data lake, удобно как warehouse. Плюс schema evolution и time travel — то, чего нет в классических data lake.w
Пример schema evolution на Silver: добавил/переименовал колонку — что произошло?	В Iceberg `ADD COLUMN`, `RENAME COLUMN`, `DROP COLUMN` — это операции над метаданными, а не перезапись данных. Например, `ALTER TABLE ... ADD COLUMN new_col INT` мгновенно добавляет колонку в схему: новые строки её видят, старые читают `NULL`, а файлы данных на диске не трогаются. Это возможно потому, что Iceberg хранит ID колонок, а не имена и позиции. Переименование `RENAME COLUMN old TO new` не ломает старые файлы — они ссылаются на тот же ID. Если я добавил колонку `new_col` в `silver.lineitem_enriched`, все 35 строк получили `NULL` в этой колонке, а

	физически в Parquet-файлах ничего не изменилось.
Пример скрипта восстановления данных из снапшота (time travel)	Iceberg хранит снапшоты — снимки состояния таблицы на момент каждого изменения данных. Можно прочитать таблицу на любой момент в прошлом: `SELECT * FROM bench.silver.lineitem_enriched FOR VERSION AS OF 1234567890;` где `1234567890` — ID снапшота. Список снапшотов: `SELECT * FROM bench.silver."lineitem_enriched$snapshots";`. Это полезно для отката ошибок, и воспроизводимости. Например, если после INSERT в Silver случайно испортились данные, можно прочитать таблицу на снапшоте до INSERT и восстановить состояние.

6.2. Подключить инструмент к lakehouse:

Я выбрал Metabase по следующим причинам:
Он самый простой в развёртывании: одна команда docker run, работает через 2–3 минуты в отличие от какого нибудь Superset, где нужно где-то 10 команд. Еще потому что он бесплатный и без лицензионных ограничений . У него есть совместимый драйвер Trino — Starburst, с которым подключение сработало.
Минимальные требования — 1 ГБ RAM, работает на ноутбуке, не конкурирует с Trino за память.
Альтернативы: Superset мощнее, но сложнее в настройке; DataLens — облако, Trino не поддерживает напрямую (только через CSV); Grafana проще, но не в списке. Для учебной задачи Metabase — оптимальный выбор по простоте, скорости и совместимости.

Скрипт развёртывания Metabase:
Metabase — open-source BI для визуализации данных lakehouse.
Порт 3000 — веб-интерфейс.
Контейнер в той же сети, что и Trino (lakehouse-net), чтобы видеть его по имени.

docker run -d --name metabase --network lakehouse-net -p 3000:3000 metabase/metabase

Мои действия — как подключился к lakehouse

1) Запуск Metabase.

Выполнил `docker run -d --name metabase -p 3000:3000 metabase/metabase`.
Контейнер поднялся, Metabase доступен на http://localhost:3000.

2) Создание администратора.

При первом входе Metabase попросил создать аккаунт:
Email: `admin@example.com`
Password: Не скажу ;)
Имя: Mateo645

3) Добавление Trino как data source.

Через `Settings → Admin settings → Databases → Add a database`:
Database type: Starburst (Trino) — совместимый драйвер.
Host: `host.docker.internal`
Port: `8080`
Catalog: `bench`
Schema: `bronze`
Username: `trino`

4) Синхронизация схемы.

После сохранения Metabase автоматически синхронизировал схему `bench.bronze` и увидел три таблицы (См. рисунок 6):
- `customer`
- `lineitem`
- `orders`

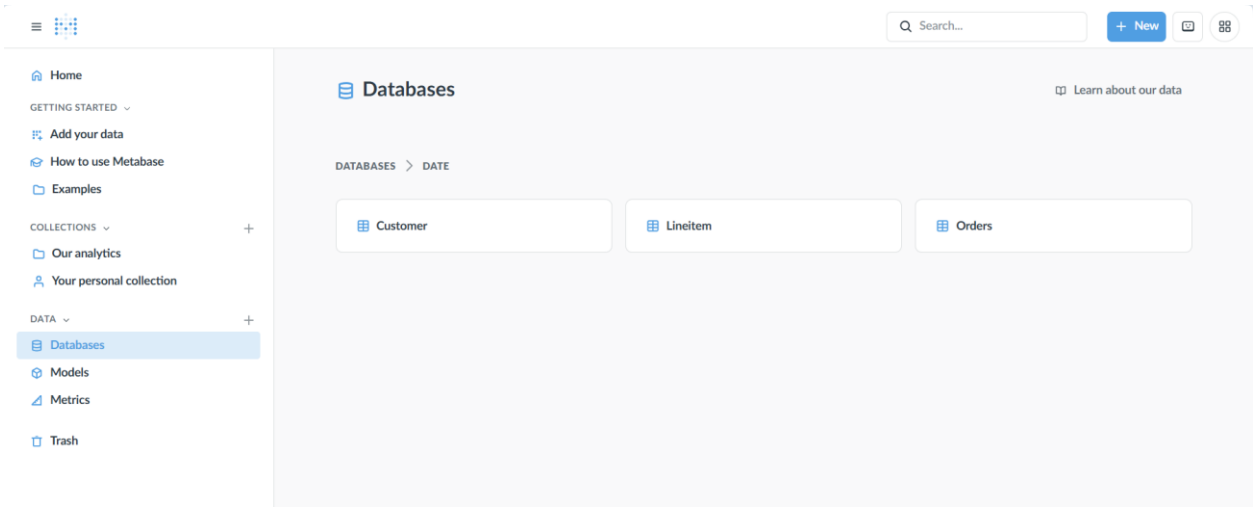


Рисунок 6 Metabase с тремя таблицами

5) Просмотр данных Bronze.
Кликнул на `lineitem` — Metabase показал первые 2000 строк сырых данных из Bronze-слоя. Это подтверждает, что BI-инструмент напрямую читает lakehouse через Trino.

6) Построение чарта.
- Добавил метрики: `Count` и `Sum of Quantity`.
- Group by: `Returnflag`.
- Visualization: Bar chart.
- Сохранил как «Count and Sum of Quantity by Returnflag».

Результат: Metabase видит данные Bronze и строит по ним визуализации — без выгрузки, копирования или промежуточного хранилища. См. рисунок 7.

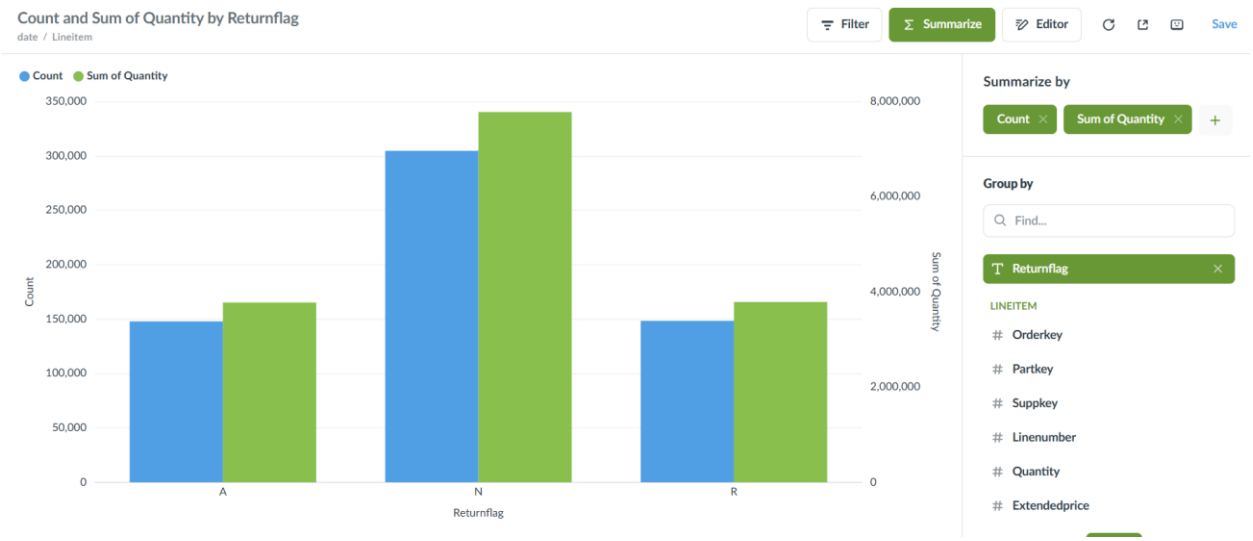


Рисунок 7 Дашборд в Metabase с данными Bronze

ВЫВОД

В ходе практической работы я построил медальон Bronze, потом медальон Silver и затем Gold и убедился, что слои должны строиться строго снизу вверх из одного масштаба. Про хранение понял, что Iceberg поверх S3 (MinIO) даёт ACID, SQL и schema evolution при стоимости обычного объектного хранилища — это дешевле классического warehouse и надёжнее «голого» data lake. Про сжатие выяснил, что на малых объёмах (14 МБ) разница между GZIP и ZSTD по размеру незаметна, а по скорости нестабильна; для продакшена на больших данных я бы выбрал ZSTD (тот же размер, меньше нагрузка на CPU), а GZIP оставил бы для архивов. Мой выбор — Parquet + GZIP и один масштаб на всех слоях — оптимален для учебного объёма sf0.1: Parquet — колоночный стандарт с поддержкой всех инструментов (Trino, Spark, Metabase), GZIP — компромисс между сжатием и скоростью, а отказ от партиционирования оправдан малым размером таблиц (< 15 МБ). По сравнению с альтернативами CSV — нет схемы и сжатия, ORC — меньше экосистема, ZSTD — избыточен на малых объёмах, Superset — сложнее в настройке, чем Metabase, моя конфигурация даёт баланс простоты, производительности и совместимости без потери функциональности.